Computer Architecture 219

Tutorial 11

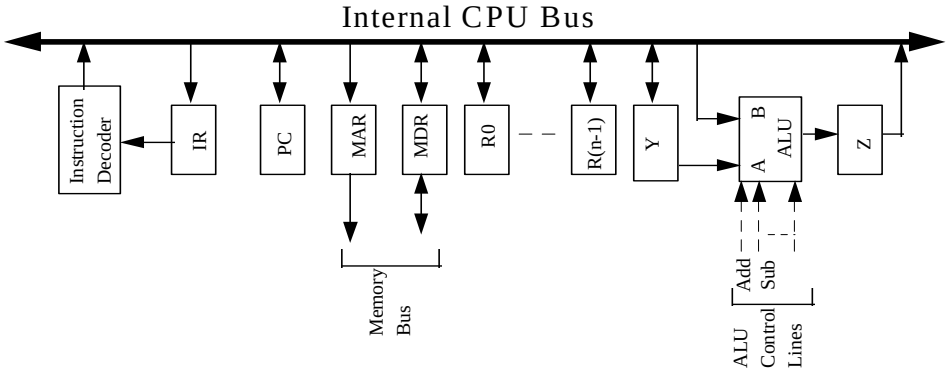
1. Your ALU can add its two input registers and it can logically complement the bits of either input register, but it cannot subtract. Numbers are to be stored in two's complement representation. List the micro-operations your control unit must perform to cause a subtraction.

2. Show the sequence of micro-operations for each of the following instructions or operations, where X is an address operand (of a variable or instruction in memory):

a. Load X into Accumulator
b. Store Accumulator to X
c. Add X to Accumulator
d. Boolean AND of X and Accumulator
e. Jump to X
f. Complement Accumulator

Assume that the fetch cycle has completed, ie write micro-operations for the execute phase only.

3. Assume that the propagation delay along the bus and through the ALU of the CPU given in the figure below are 20 ns and 100 ns, respectively. The time required for a register to load/copy data (eg from the bus) is called the register setup time and is 10 ns. What is the time that must be allowed for: (a) transferring data from one register to another, (b) incrementing the program counter.



4. Write the sequence of micro-orders, not micro-operations, for the execution phase of the following instructions on the CPU given by the figure above:

a) BNE R1, R2, 200
(branch if R1 and R2 not equal, an offset of 200 relative to PC)

b) LD R1, 200(R0)
(load from memory address given in base-displacement format)

Use the following sequence, which is for the instruction fetch, as an example:

PC_{out} ; MAR_{in} ; Read ; Increment ; Z_{in}
Wait for MFC ; Z_{out} ; PC_{in}
MDR_{out} ; IR_{in}

Tutorial 11 Solutions

I need students to understand the difference between micro-operations, micro-orders, and micro-instructions.

micro-operations: the atomic operations of a processor (multiple micro-operations required for the implementation of an instruction), eg PC ← (PC) + 1 which is the PC getting incremented
micro-orders: the individual control signals that implement a micro-operation (multiple micro-orders required for the implementation of a micro-operation), eg $R1_{out}$ which is the control signal that triggers the latch which copies the contents of R1 to the internal bus.
micro-instructions: the microprogramming equivalent of an instruction; the bits that encode the control signals to be set in a cycle

Question 1

Consider the instruction SUB R1, X, which subtracts the contents of location X from the contents of register R1, and places the result in R1.

t_1: MAR ← ($IR_{address}$)
t_2: MBR ← Memory
t_3: MBR ← Complement(MBR)
t_4: MBR ← Increment(MBR)
t_5: R1 ← (R1) + (MBR)

This works because the two's complement of a number is found by inverting it's bits (complementing it) and adding 1.

Question 2

LOAD AC:

t_1: MAR ← ($IR_{address}$)
t_2: MBR ← Memory
t_3: AC ← (MBR)

STORE AC:

t_1: MAR ← ($IR_{address}$)
t_2: MBR ← (AC)
t_3: Memory ← (MBR)

ADD AC:

t_1: MAR ← ($IR_{address}$)
t_2: MBR ← Memory
t_3: AC ← (AC) + (MBR)

AND AC:

t_1: MAR ← ($IR_{address}$)
t_2: MBR ← Memory
t_3: AC ← (AC) AND (MBR)

JUMP:

t_1: PC $\leftarrow (IR_{address})$

Complement AC:

t_1: AC $\leftarrow (\overline{AC})$

Question 3

(a) Time required = propagation time + copy time = 30 ns.

(b) Incrementing the program counter involves two steps:

$Z \leftarrow (PC) + 1$
$PC \leftarrow (Z)$

The first step requires 20 + 100 + 10 = 130 ns.
The second step requires 30 ns (see part a)
Total time = 160 ns.

Question 4

Note: the example micro-orders for the Fetch cycle use the ALU "Increment" operation. This is sometimes implemented (eg in the examples in the lecture notes) as:

Clear Y ; Set Carry-IN to ALU ; Add

a)

t1: $R1_{out}$; Y_{in}
t2: $R2_{out}$; Subtract ; Z_{in}
if not ZERO (condition flag) then
t3: PC_{out} ; Y_{in}
t4: $IR_{imm,out}$; Add ; Z_{in}
t5: Z_{out} ; PC_{in}
endif

Though technically, the PC has already been incremented in the fetch cycle so this would branch to PC+201. The value of the immediate operand would be set to allow for this.

b)
The instruction does $R1 \leftarrow (200 + (R0))$

t1: $R0_{out}$; Y_{in}
t2: $IR_{imm,out}$; Add ; Z_{in}
t3: Z_{out} ; MAR_{in} ; Read
t4: Wait for MFC
t5: MDR_{out} ; $R1_{in}$